🎮 The Maze Game

Complete Documentation & Build Guide — ADS S2 Project

📋 Table of Contents

1. Project Overview
2. Project Architecture (Modularity)
3. File: map.h — Configuration & Prototypes
4. File: map.c — Maze Logic & Display
5. File: main.c — Game Loop & Player Control
6. The Maze Generation Algorithm (DFS)
7. How to Build from Scratch
8. How to Compile & Run

1. Project Overview

The Maze Game is a console-based game written in **C** for Windows. The player navigates through a randomly generated maze using the keyboard (W/A/S/D), trying to reach the exit. Each level generates a harder maze (bigger grid, more walls). The game features sound effects, a scoring system, and flicker-free rendering.

Key Features

- **True Maze Generation:** Uses the Recursive Backtracker (DFS) algorithm — every maze is guaranteed to be solvable.
- **Infinite Levels:** The maze grows larger each level, increasing the difficulty.
- **Sound Effects:** Movement beeps, wall collision buzzes, and a victory melody using `Beep()` .
- **Flicker-Free Display:** Uses `SetConsoleCursorPosition()` instead of `system("cls")` for smooth rendering.
- **Colored Output:** Uses ANSI escape codes for colored walls, player, and exit.
- **Score Tracking:** Score increases based on the level cleared.

Libraries Used

Library	Purpose
`<stdio.h>`	Standard input/output (`printf`)
`<stdlib.h>`	Random numbers (`rand` , `srand`), `system("cls")`
`<time.h>`	Seed for random number generation
`<windows.h>`	Console API (`SetConsoleCursorPosition` , `Beep` , `Sleep`)
`<conio.h>`	`_getch()` — Read a key without requiring Enter

2. Project Architecture (Modularity)

The project follows a **modular design**, splitting the code into separate files by responsibility. This is a best practice in C programming and is expected in ADS coursework.

```
┌──────────────┐
│   map.h      │  ← Configuration (constants, colors, prototypes)
└──────┬───────┘
       │ #include "map.h"
┌──────┴───────┐
│   map.c      │  ← Maze logic (generation, display, cursor reset)
└──────┬───────┘
       │ linked during compilation
┌──────┴───────┐
│   main.c     │  ← Game loop (levels, input, movement, sounds)
└──────────────┘
```

File	Role	Contains
`map.h`	Header (Declarations)	Constants, color macros, function prototypes
`map.c`	Implementation (Logic)	`shuffle()` , `carveMaze()` , `initMap()` , `resetCursor()` , `displayMap()`
`main.c`	Entry Point (Control)	`main()` — level loop, input handling, score, sounds

3. File: map.h — Configuration & Prototypes

.h map.h

This is the **header file** — it contains no logic, only declarations that are shared between `map.c` and `main.c` .

Constants

Constant	Value	Purpose
`MAX_SIZE`	31	Maximum maze dimensions. Must be odd.
`EMPTY`	0	Represents a walkable corridor cell
`WALL`	1	Represents a solid wall cell
`PLAYER`	2	The player's current position
`EXIT`	3	The maze exit (goal)

Color Macros

These are ANSI escape sequences that change the text color in the terminal:

Macro	Code	Used For

RED	\033[31m	Walls
GREEN	\033[32m	Player
BLUE	\033[34m	Border frame
MAGENTA	\033[35m	Exit door
YELLOW	\033[33m	Score & controls text
CYAN	\033[36m	Game title
RESET	\033[0m	Reset color to default

Function Prototypes

```
void initMap(int map[MAX_SIZE][MAX_SIZE], int row, int clm);
void displayMap(int map[MAX_SIZE][MAX_SIZE], int row, int clm, int level, int score);
```

These prototypes tell the compiler that these functions exist (defined in `map.c`) so `main.c` can call them.

4. File: map.c — Maze Logic & Display

.c **map.c**

This file contains all the maze generation and rendering logic. It has 5 functions:

4.1 — `shuffle(int array[], int n)`

> **Purpose:** Randomly rearranges (shuffles) an array of integers.
> **Why:** Used to randomize the order of directions (up/down/left/right) so each maze is unique.

```
void shuffle(int array[], int n) {
    for (int i = 0; i < n; i++) {
        int j = rand() % n;    // Pick a random index
        int t = array[i];      // Swap array[i] and array[j]
        array[i] = array[j];
        array[j] = t;
    }
}
```

How it works: For each element in the array, it picks a random position and swaps them. This is a variant of the Fisher-Yates shuffle.

4.2 — `carveMaze(map, x, y, row, clm)`

> **Purpose:** The core maze generation algorithm. Uses **Recursive Backtracking (DFS)** to carve corridors through a grid of walls.
> **Algorithm:** Depth-First Search (DFS) — a key topic in your ADS course.

```
void carveMaze(int map[MAX_SIZE][MAX_SIZE], int x, int y, int row, int clm) {
    // Direction vectors: each jumps 2 cells
    int dx[] = {0, 0, -2, 2};
    int dy[] = {-2, 2, 0, 0};
    int dirs[] = {0, 1, 2, 3};
    shuffle(dirs, 4);  // Randomize direction order

    for (int i = 0; i < 4; i++) {
        int nx = x + dx[dirs[i]];
        int ny = y + dy[dirs[i]];

        // Check: in bounds? and target cell still a wall?
        if (nx > 0 && nx < row-1 && ny > 0 && ny < clm-1 && map[nx][ny] == WALL) {
            map[x + dx[dirs[i]]/2][y + dy[dirs[i]]/2] = EMPTY;  // Break wall between
            map[nx][ny] = EMPTY;                  // Mark destination as corridor
            carveMaze(map, nx, ny, row, clm);  // Recurse from new cell
        }
    }
}
```

> **Key Concept — Why jump 2 cells?**
> In the initial grid, every cell is a WALL. The algorithm treats **odd-indexed cells** (1,1), (1,3), (3,1)... as potential corridors, and the **even-indexed cells** between them as the walls separating corridors. When we "carve" from (1,1) to (1,3), we also break the wall at (1,2). This produces a perfect maze with proper walls between every corridor.

Visual example of carving:

```
BEFORE carving:                AFTER carving from (1,1) to (1,3):

  ██████████                     ██████████
  ■■  ■■■■■■■■                   ■■         ■■■■
  ■■■■■■■■■■                     ■■■■■■■■■■
  ■■■■■■■■■■                     ■■■■■■■■■■

  (1,1) is empty                 (1,1), (1,2), (1,3) are empty
  Everything else = WALL          Wall between them is broken
```

4.3 — `initMap(map, row, clm)`

> **Purpose:** Initializes the maze by filling it with walls, then carving corridors, then placing the player and exit.

```
void initMap(int map[MAX_SIZE][MAX_SIZE], int row, int clm) {
    // Step 1: Fill entire grid with walls
    for (...) map[i][j] = WALL;

    // Step 2: Start carving from (1,1)
    map[1][1] = EMPTY;
    carveMaze(map, 1, 1, row, clm);
```

```
    // Step 3: Place Player at top entrance, Exit at bottom
    map[0][1] = PLAYER;
    map[row-1][clm-2] = EXIT;
}
```

3 steps:

1. 1 Fill the entire grid with `WALL` — a solid block of walls.
2. 2 Call `carveMaze()` starting from cell (1,1) to carve corridors using DFS.
3. 3 Place the `PLAYER` at position (0,1) (top entrance) and `EXIT` at (row-1, clm-2) (bottom exit).

4.4 — `resetCursor()`

> **Purpose:** Moves the console cursor back to position (0,0) **without clearing the screen**. This allows us to overwrite the previous frame, eliminating screen flicker.

```
void resetCursor() {
    HANDLE hOut = GetStdHandle(STD_OUTPUT_HANDLE);  // Get console handle
    COORD pos = {0, 0};                              // Target position (top-left)
    SetConsoleCursorPosition(hOut, pos);             // Move cursor there
}
```

Why not `system("cls")`? `system("cls")` clears the entire screen (makes it black for a split second) then redraws everything. This causes visible flickering. `resetCursor()` just moves the cursor to (0,0) and overwrites the old text with new text — the screen never goes blank, so it looks perfectly smooth.

4.5 — `displayMap(map, row, clm, level, score)`

> **Purpose:** Renders the entire game screen: title, score, maze grid with colored borders, and control instructions.

Output structure:

```
═══ THE MAZE GAME ═══
Level: 1    Score: 0

╔═══════════════════╗
║═══════════════════╕    ← GREEN = Player
│   ▄▄   ▄▄         │     ← RED = Walls
│        ▄▄▄▄▄▄     │     ← Spaces = Corridors
╘═══▄▄▄▄▄▄▄▄▄▄   ▄▄═╛    ← MAGENTA = Exit
╚═══════════════════╝

[W/A/S/D] : Move   |   [Esc] : Quit
```

Each cell takes **2 characters wide** (▄▄ for walls, for empty) to maintain a square aspect ratio in the terminal.

5. File: main.c — Game Loop & Player Control

.c main.c

5.1 — Initialization

```
SetConsoleOutputCP(CP_UTF8);  // Enable UTF-8 for special chars (╬, ╔, etc.)
srand((unsigned)time(NULL));   // Seed random with current time

int level = 1, score = 0;
int row = 9, clm = 9;         // Starting maze size (MUST be odd)
```

5.2 — The Level Loop (Outer `while`)

Each iteration of this loop represents one level:

1. Generate a new maze with `initMap()`
2. Reset the player position to the top entrance (0, 1)
3. Enter the Game Loop
4. When the player wins, increase difficulty and `break` to the next iteration

5.3 — The Game Loop (Inner `while`)

Each iteration of this loop represents one frame:

1. **Draw:** Call `displayMap()` to render the current state
2. **Check Win:** If player position == exit position → play victory sound, add score, increase level
3. **Read Input:** `_getch()` waits for a keypress (W/A/S/D or Esc)
4. **Calculate Next Position:** Based on the key pressed, compute `nx`, `ny`

5. **Validate Move:** Check if the next position is within bounds and not a wall
6. **Update Map:** Clear old position, set new position, play appropriate sound

5.4 — Input Handling

Key	Action	Direction Vector
`W`	Move Up	nx = px - 1
`S`	Move Down	nx = px + 1
`A`	Move Left	ny = py - 1
`D`	Move Right	ny = py + 1
`Esc` (27)	Quit Game	return 0

5.5 — Sound Effects

Event	Sound	Code
Movement	Short high beep	`Beep(800, 30)`
Wall Collision	Low buzz	`Beep(150, 80)`
Level Clear	Victory melody (C-E-G-C)	`Beep(523,100); Beep(659,100); ...`
Game Over (Esc)	Sad descending tone	`Beep(400,300); Beep(300,500)`

5.6 — Difficulty Scaling

After each level is cleared:

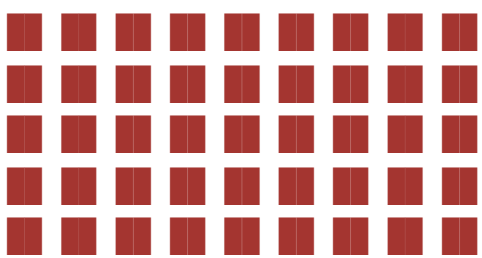
- `row += 2` and `clm += 2` — Maze grows (but stays odd)
- Score increases by `level * 10` — Higher levels give more points
- Maximum size is capped at `MAX_SIZE` (31×31)

6. The Maze Generation Algorithm (DFS)

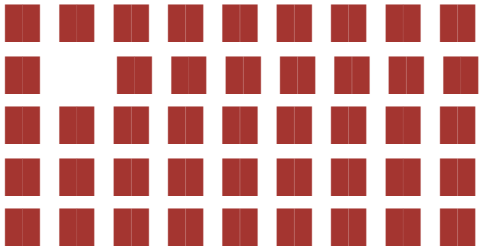
This section explains the **Recursive Backtracker** algorithm step-by-step. This is a DFS-based algorithm, which is a core topic in your ADS (Algorithms & Data Structures) course.

Step-by-Step Walkthrough

1 **Start with a full grid of walls:**

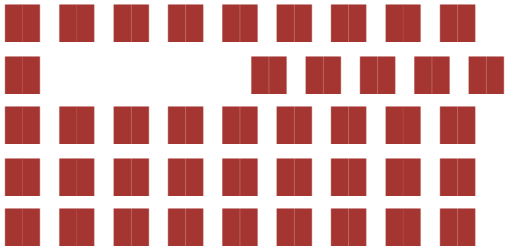


2 **Mark cell (1,1) as empty:**



← (1,1) is now empty

3 **Pick a random unvisited neighbor 2 cells away. Break the wall between them:**



← Carved right: (1,1)→(1,2)→(1,3)

4 **Repeat recursively from the new cell until all cells are visited. When stuck (no unvisited neighbors), backtrack.**

> **Why dimensions must be ODD:**
> The algorithm uses odd-indexed cells (1,1), (1,3), (3,1), (3,3)... as corridor cells, and even-indexed cells as walls

> between them. If the grid were even-sized (e.g., 8×8), the border wouldn't align properly. Odd sizes (9, 11, 13...) guarantee a clean outer wall and proper corridor spacing.

7. How to Build from Scratch

If you want to recreate this project from zero, follow these steps:

1 Create the project folder

```
mkdir maze_game
cd maze_game
```

2 Create `map.h`

Define your constants (`MAX_SIZE` , `WALL` , `EMPTY` , `PLAYER` , `EXIT`), color macros, and function prototypes.

3 Create `map.c`

Implement the functions in this order:

1. `shuffle()` — Simple array shuffler (needed by carveMaze)
2. `carveMaze()` — The DFS recursive backtracker
3. `initMap()` — Fill grid with walls, call carveMaze, place player/exit
4. `resetCursor()` — Windows API cursor repositioning
5. `displayMap()` — Print the entire game screen

4 Create `main.c`

Build the game loop:

1. Initialize random seed, level, score, and starting dimensions
2. Outer `while(1)` — Level loop (generate maze, reset player)
3. Inner `while(1)` — Game loop (draw, check win, read input, move)
4. Add sound effects with `Beep(frequency, duration)`
5. Add difficulty scaling after each level clear

5 Compile and test

```
gcc main.c map.c -o maze.exe
.\maze.exe
```

8. How to Compile & Run

Using GCC directly:

```
gcc main.c map.c -o maze.exe
.\maze.exe
```

Using the Makefile:

```
make run       # Compile and run
make clean     # Delete compiled files
```

> **Important:** You must compile **both** `main.c` and `map.c` together. If you only compile `main.c`, you'll get "undefined reference" errors because the linker can't find `initMap()` and `displayMap()`.